

One Graph, Many Facades

How a normalized
graph store — the same
idea behind your
GraphQL client cache
— makes AI-agent
evaluations
deterministic, offline,
and **safe.**

- A talk for GraphQL +
AI-agent builders
- Conference edition

— WHY AGENT EVALS ARE HARD

Agents read *and write* live backends

So evals break two ways: the data **moves** underneath you (a pass yesterday fails today), and every run **mutates production** — so the write flows that matter most go untested.



non-deterministic



pollutes prod



needs the network

— THE IDEA

Mock with a Relay-like store

A GraphQL client cache doesn't keep nested blobs — it **normalizes** every entity into one flat record keyed by ID, with relationships as references. Use that same store to **serve the mock data** in an eval. And because it's just a graph of entities, it isn't GraphQL-only — the very same store backs every other interface too.

Flat, linked records — authored once

A QUERY THE AGENT RUNS

```
query {  
  user(id: "u1") {  
    name  
    reports { name }  
  }  
}
```



THE STORE THAT ANSWERS IT

```
STORE = {  
  "client:root": { # entry point  
    'user(id:"u1")': {__ref:"u1"} },  
  "u1": {User, name:"Ada",  
    reports:{__refs:["u2","u3"]}},  
  "u2": {User, name:"Grace"},  
  "u3": {User, name:"Alan"},  
}
```

Two primitives: **__ref** (one link) · **__refs** (a list). Reading is a graph walk.

— QUERIES & MUTATIONS

Add logic with a @decorator

MOCK A QUERY

```
@graphql_query("currentUser")
def current_user(store, args):
    return {"__ref": "u1"}
```

MOCK A MUTATION — NEVER TOUCHES PROD

```
@graphql_mutation("closeTask")
def close_task(store, args):
    store.set_field(args["id"], "state", "CLOSED")
    return {"task": {"__ref": args["id"]}}
```

Static reads live as data; anything with logic gets a tiny resolver. A mutation **writes the store**, so the next query reads it back — and **production is never written**.

— ONE STORE, EVERY INTERFACE

GraphQL was just the first facade



Same records, same decorators. A write via one facade is a read via any other — so **multi-step** workflows just work.

— TAKEAWAY

Your client cache was a mocking engine all along

- ◆ A **Relay-like normalized store** serves your mock GraphQL data.
- ◆ The **same store** covers CLI, MCP tools, and SQL — one graph, many facades.
- ◆ Queries & mutations are just **@decorated resolvers** — writes hit the store, never prod.

Deterministic, offline, safe agent evals — built on the graph you already have.

